

LC 2332 - Latest Time to Catch a Bus

Companies: Amazon, Zoho

Problem Statement & Rules

I/P → departure time of buses, arrival time of passengers, capacity of bus

- If `<= capacity` passengers are waiting for a bus, they will all get on bus.
- If `> capacity` are waiting then the ones who arrived earliest will get on bus.

O/P: latest time you may arrive at bus station to catch a bus, cant arrive at same time as other passengers

Examples

Eg 1: `[10, 20]` , pass = `[2, 17, 18, 19]` , cap = `2`

- then bus 1 takes 2
- bus 2 would have took 17, 18, but you must arrive at 16, to get a seat

Eg 2: `[20, 30, 10]` , pass `[19, 13, 26, 4, 25, 11, 21]` , cap = `2`

- let first sort
- `[10, 20, 30]` , `[4, 11, 13, 19, 21, 25, 26]`
- now there are 3 passengers but we can take only 2, so 19 is left
- 19, 21 would have hopped, so you need to get at 20

Observation & Approach

now agar 20 pe bus mil gayi to 19, 18, 17... pe bhi mil jayegi

hence **MONOTONIC + MAX** → **BINARY**

However this question can be done without any binary search using just simulation.

Algorithm: JUST SORT

S1: maintain a passenger idx, to keep track of who is first in line

S2: whenever `passidx < p && passengers[passidx] <= buses[bus] && capu < capacity`. Then add a passenger to the bus

S3: if while processing you arrive at the last bus, and capacity isnt yet filled then ans is basically the departure time of bus otherwise if it's filled, it arrival time of the passenger who took the bus last. i.e `passengers[passidx - 1]`

S4: no 2 passenger can arrive at same time so we keep decreasing ans until its not found in array

Complexity

$O(p \log p + b \log b)$

Implementation

```
class Solution {
public:
    int latestTimeCatchTheBus(vector<int>& buses, vector<int>& passengers,
int capacity) {

        sort(buses.begin(), buses.end());
        sort(passengers.begin(), passengers.end());

        int b = buses.size();
```

```

int p = passengers.size();
int passidx = 0;

// Simulate boarding for every bus
for(int bus = 0; bus < b; bus++){

    int capu = 0; //initially har bus 0 seat filled ke saath aayegi

    while(passidx < p && passengers[passidx] <= buses[bus] && capu
< capacity){

        passidx++;
        capu++;
    }

    //last bus
    if(bus == b - 1){

        int ans;
        //agar bus me seat bacchi he to seedhe bus ke departure
time me aao
        if(capu < capacity){
            ans = buses[bus];
        }
        //agar full ho gayi to last passenger jab aaya uske time pe
aao
        else{
            ans = passengers[passidx - 1];
        }

        //uske time pe nhi aa skte
        while(binary_search(passengers.begin(), passengers.end(),
ans)){
            ans--;
        }

        return ans;
    }
}

return -1;
};

```

